

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 200

/*
 * Process escape sequences.
 * Supported:
 * \ -> backslash
 * \ -> escaped space
 * \n -> newline
 * \t -> tab
 * \& -> &
 * \| -> |
 * \< -> <
 * \> -> >
 * \$ -> $
 * \\ -> \
 */

int processEscapes(const char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1) {
        /* Normal character */
        if (input[i] != '\\') {
            output[j++] = input[i];
            i++;
        }
        /* Escape sequence */
        else {
            i++;
            /* Backslash at end of input */
            if (input[i] == '\0') {
                printf("Error: Incomplete escape sequence.\n");
                return 0;
            }
            switch (input[i]) {
                case 'n':
                    output[j++] = '\n';
                    break;
                case 't':
                    output[j++] = '\t';
                    break;
                case ' ':
                    output[j++] = ' ';
                    break;
```

```
GNU nano 8.7.1
                case '&':
                    output[j++] = '&';
                    break;
                case '|':
                    output[j++] = '|';
                    break;
                case '<':
                    output[j++] = '<';
                    break;
                case '>':
                    output[j++] = '>';
                    break;
                case '$':
                    output[j++] = '$';
                    break;
                case '\\':
                    output[j++] = '\\';
                    break;
                default:
                    /* Preserve unknown escaped characters. */
                    output[j++] = input[i];
                    break;
            }
            i++;
        }
    }
    output[j] = '\0';
    return 1;
}

/* Display characters including special characters */
void displayOutput(const char *output) {
    printf("\nProcessed Output:\n");
    printf("-----\n");
    for (int i = 0; output[i] != '\0'; i++) {
        if (output[i] == '\n')
            printf("[NEWLINE]\n");
        else if (output[i] == '\t')
            printf("[TAB]");
        else
```

```

GNU nano 8.7.1

/* Display characters including special characters */
void displayOutput(const char *output) {

    printf("\nProcessed Output:\n");
    printf("-----\n");

    for (int i = 0; output[i] != '\0'; i++) {

        if (output[i] == '\n')
            printf("[NEWLINE]\n");

        else if (output[i] == '\t')
            printf("[TAB]");

        else
            printf("%c", output[i]);

    }

    printf("\n-----\n");
}

/* Validate parser output */
void validateOutput(const char *output) {

    if (strlen(output) == 0) {
        printf("Parser Output: Empty.\n");
    }
    else {
        printf("Parser Output: Valid.\n");
    }
}

int main() {

    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Escape Sequence Parser\n");
    printf("-----\n");

    printf("Enter input: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline added by fgets */
    input[strcspn(input, "\n")] = '\0';

    /* Process escape sequences */
    if (!processEscapes(input, output)) {
        return 1;
    }

    /* Display result */
    displayOutput(output);

    /* Validate result */
    validateOutput(output);

    return 0;
}

```

Result:

```

abhinav@Abhinav:~/SKILL$ nano Skill6a.c
abhinav@Abhinav:~/SKILL$ gcc Skill6a.c -o s6a
abhinav@Abhinav:~/SKILL$ ./s6a
Escape Sequence Parser
-----
Enter input: Hello\ World

Processed Output:
-----
Hello World
-----
Parser Output: Valid.
abhinav@Abhinav:~/SKILL$ ./s6a
Escape Sequence Parser
-----
Enter input: Hello&\World\|Test

Processed Output:
-----
Hello&World|Test
-----
Parser Output: Valid.
abhinav@Abhinav:~/SKILL$

```

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create child process
    pid = fork();

    if (pid < 0) {

        // Handle fork error
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {

        // Child process
        printf("\nChild Process Created\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        // Execute program with arguments
        printf("Executing ls command...\n");

        execlp("ls", "ls", "-l", NULL);

        // This executes only if execlp fails
        perror("Execution failed");
        exit(1);
    }

    else {

        // Parent process
        printf("\nParent waiting for child...\n");

        // Wait for child process
        if (waitpid(pid, &status, 0) == -1) {

            perror("waitpid failed");
            return 1;
        }

        // Check child exit status
        if (WIFEXITED(status)) {

            printf("\nChild process completed.\n");
            printf("Child exit status: %d\n",
                WEXITSTATUS(status));
        }
    }
}
```

```
else {

    // Parent process
    printf("\nParent waiting for child...\n");

    // Wait for child process
    if (waitpid(pid, &status, 0) == -1) {

        perror("waitpid failed");
        return 1;
    }

    // Check child exit status
    if (WIFEXITED(status)) {

        printf("\nChild process completed.\n");
        printf("Child exit status: %d\n",
            WEXITSTATUS(status));
    }

    else {

        printf("Child process terminated abnormally.\n");
    }
}

printf("\nParent Process Completed\n");

return 0;
}
```

Result:

```
abhinav@Abhinav: ~/ × + v
abhinav@Abhinav: ~/SKILL$ nano Skill6b.c
abhinav@Abhinav: ~/SKILL$ gcc Skill6b.c -o s6b
abhinav@Abhinav: ~/SKILL$ ./s6b
Parent Process Started
Parent PID: 1775

Parent waiting for child...

Child Process Created
Child PID: 1776
Parent PID: 1775
Executing ls command...
total 304
-rwxr-xr-x 1 abhinav abhinav 16216 Oct 3 16:19 S2a
-rwxr-xr-x 1 abhinav abhinav 16472 Oct 3 16:24 S2b
-rwxr-xr-x 1 abhinav abhinav 16776 Oct 3 16:35 S3a
-rwxr-xr-x 1 abhinav abhinav 16360 Oct 3 16:41 S3b
-rw-r--r-- 1 abhinav abhinav 3206 Oct 3 17:18 Skill6a.c
-rw-r--r-- 1 abhinav abhinav 316 Aug 24 06:01 Skill1.c
-rw-r--r-- 1 abhinav abhinav 938 Oct 3 16:19 Skill2a.c
-rw-r--r-- 1 abhinav abhinav 2496 Oct 3 16:24 Skill2b.c
-rw-r--r-- 1 abhinav abhinav 4725 Oct 3 16:34 Skill3a.c
-rw-r--r-- 1 abhinav abhinav 3690 Oct 3 16:41 Skill3b.c
-rw-r--r-- 1 abhinav abhinav 3639 Oct 3 16:53 Skill4a.c
-rw-r--r-- 1 abhinav abhinav 3998 Oct 3 16:59 Skill4b.c
-rw-r--r-- 1 abhinav abhinav 2959 Oct 3 17:07 Skill5a.c
-rw-r--r-- 1 abhinav abhinav 4844 Oct 3 17:11 Skill5b.c
-rw-r--r-- 1 abhinav abhinav 3206 Oct 3 17:21 Skill6a.c
-rw-r--r-- 1 abhinav abhinav 1403 Oct 3 17:25 Skill6b.c
-rwxr-xr-x 1 abhinav abhinav 16640 Oct 3 16:59 a.out
-rwxr-xr-x 1 abhinav abhinav 16424 Oct 3 16:54 s4a
-rwxr-xr-x 1 abhinav abhinav 16640 Oct 3 16:59 s4b
-rwxr-xr-x 1 abhinav abhinav 16336 Oct 3 17:05 s5a
-rwxr-xr-x 1 abhinav abhinav 16416 Oct 3 17:11 s5b
-rwxr-xr-x 1 abhinav abhinav 16336 Oct 3 17:22 s6a
-rwxr-xr-x 1 abhinav abhinav 16344 Oct 3 17:25 s6b
-rwxr-xr-x 1 abhinav abhinav 16760 Sep 29 03:55 sh
-rw-r--r-- 1 abhinav abhinav 4968 Sep 29 03:55 shell_history.c
-rwxr-xr-x 1 abhinav abhinav 16176 Aug 24 05:55 skill
-rw-r--r-- 1 abhinav abhinav 3639 Oct 3 16:54 skill4_tokenizer.c

Child process completed.
Child exit status: 0

Parent Process Completed
abhinav@Abhinav: ~/SKILL$
```